

G17

# Fine-Tune a Small LLM with LoRA / PEFT + Simple Agent

---

Academic Project Report

## TEAM MEMBERS

Mayank Garg — 2023UAI1797

Rishabh Kumar — 2023UAI1803

## DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA ENGINEERING

B.Tech , Academic Year 2025–2026

## Table of Contents

---

1. Problem Statement
2. Objectives
3. System Architecture
4. Deep Learning Concepts Applied
5. Implementation Details
6. Results and Evaluation
7. Challenges Faced and Solutions
8. Conclusion
9. References

# 1. Problem Statement

Modern software development workflows increasingly demand intelligent code assistance. While frontier models such as GPT-4 and Claude excel at complex coding tasks, they are hosted services with associated costs, latency, and data-privacy concerns. Individual developers, students, and researchers who operate in resource-constrained environments — local laptops, Kaggle notebooks, or university compute clusters — are effectively excluded from these capabilities.

Smaller open-source models (1–3 billion parameters) offer a feasible alternative, but their out-of-the-box performance on repository-oriented coding tasks is insufficient. Instruction-following quality, code-aware reasoning, and the ability to produce structured responses without verbose hallucinations all degrade at small model scales. The core challenge is therefore:

|                       |                                                                                                                                                                                                               |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Core Challenge</b> | How can a compact open model be efficiently adapted — using minimal compute — to serve as a practical, reliable coding assistant for repository-aware tasks, while remaining deployable on consumer hardware? |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

This project directly addresses that challenge by combining parameter-efficient fine-tuning (LoRA/QLoRA) with a focused instruction dataset and a lightweight agent wrapper, producing a locally runnable coding assistant that meaningfully outperforms its base model.

## 2. Objectives

### 2.1 Primary Goals

- Fine-tune Gemma-2B using LoRA/QLoRA on the CodeAlpaca-20k dataset within a single Kaggle T4 GPU session ( $\leq 16$  GB VRAM).
- Achieve measurable improvement over the base model on task completion rate, code correctness, and response clarity.
- Build a Python-based coding agent that wraps the fine-tuned model with file-inspection and prompt-structuring tools.
- Produce a reproducible training and evaluation pipeline documented sufficiently for peer replication.

### 2.2 Expected Outcomes

| Outcome                      | Description                                                                      |
|------------------------------|----------------------------------------------------------------------------------|
| <b>Fine-tuned Model</b>      | A Gemma-2B model with LoRA adapters.                                             |
| <b>Coding Agent</b>          | Agent that accepts repository tasks & produces structured, code-aware responses. |
| <b>Evaluation Suite</b>      | A set of coding tasks with baseline and fine-tuned model outputs for comparison. |
| <b>Reproducible Pipeline</b> | Training scripts, hyperparameter configs, and dataset preprocessing code.        |

### 3. System Architecture

The system is organised into five functional layers, each with a well-defined responsibility. This separation of concerns ensures that the core model remains interchangeable and that the surrounding application logic can be extended without retraining.

#### 3.1 Layer Overview

| LAYER           | COMPONENT                                    | TECHNOLOGY                 |
|-----------------|----------------------------------------------|----------------------------|
| Input Layer     | User Task / Prompt Interface                 | Python CLI / Notebook      |
| Context Layer   | Repository File Snippets / Directory Summary | LangChain File Tools       |
| Inference Layer | Fine-tuned Gemma-2B (LoRA adapter)           | Transformers + PEFT        |
| Output Layer    | Structured Code Response                     | Post-processor / Formatter |
| Orchestration   | Agent Loop & Tool Execution                  | LangChain / Custom Python  |

Table 1 — System Architecture Layers

#### 3.2 Inference Pipeline

At runtime, each user request traverses the following stages. The model output is kept strictly separate from tool execution, allowing the user to inspect each step independently.

| Step | Stage            | Description                                                    |
|------|------------------|----------------------------------------------------------------|
| 1    | Accept Task      | User submits a coding question or repository task              |
| 2    | Normalize Prompt | Task is structured into a templated instruction format         |
| 3    | Inject Context   | Relevant file snippets or directory summaries are prepended    |
| 4    | Tokenize Input   | Tokenizer encodes the prompt into token IDs + attention mask   |
| 5    | Model Inference  | Fine-tuned Gemma generates token predictions autoregressively  |
| 6    | Decode Output    | Token IDs are decoded back to human-readable text              |
| 7    | Post-process     | Formatter validates, strips noise, and structures the response |

Table 2 — Inference Pipeline Stages

#### 3.3 Technologies Used

- PyTorch 2.2 with CUDA backend for tensor operations and gradient computation.
- Hugging Face Transformers for model loading, tokenization, and generation.
- PEFT (Parameter-Efficient Fine-Tuning) library for LoRA adapter management.
- TRL SFTTrainer for supervised instruction fine-tuning with packing support.
- BitsAndBytes for 4-bit NF4 quantization enabling QLoRA on limited VRAM.
- LangChain for agent orchestration, tool definitions, and prompt templates.

- Kaggle Notebooks (T4 GPU) / local CUDA machine as the training environment.

## 4. Deep Learning Concepts Applied

---

### 4.1 Transformer Architecture

Gemma-2B is a decoder-only Transformer. Each layer applies multi-head self-attention followed by a feed-forward network. The attention mechanism computes  $Q \cdot K^T / \sqrt{d_k}$  softmax scores, enabling the model to attend to all previous tokens simultaneously. Rotary Positional Embeddings (RoPE) encode positional information relative to token distance rather than absolute position, improving generalisation to longer sequences.

### 4.2 Tokenization and Embeddings

Input text is tokenised by the Gemma SentencePiece tokenizer (vocabulary  $\approx 256k$  tokens) into sub-word units. Each token ID is projected through a learned embedding matrix ( $d_{\text{model}} = 2048$ ) into a dense vector representation. These vectors, combined with RoPE encodings, form the input to the first Transformer layer.

### 4.3 LoRA — Low-Rank Adaptation

Full fine-tuning of a 2B parameter model requires storing and updating every weight, which is prohibitive on limited hardware. LoRA addresses this by injecting trainable low-rank matrices  $A \in \mathbb{R}^{(d \times r)}$  and  $B \in \mathbb{R}^{(r \times k)}$  alongside the frozen pre-trained weight  $W$ . The effective weight becomes  $W' = W + BA$ . With rank  $r = 16$  applied to  $Q$  and  $V$  projection matrices, only  $\sim 1.2\%$  of total parameters are trained, dramatically reducing memory and compute.

### 4.4 QLoRA — Quantised LoRA

QLoRA combines LoRA with 4-bit NormalFloat (NF4) quantization of the frozen base model weights. The base model is stored in NF4 ( $\approx 0.5$  bytes/param) while LoRA adapters remain in BF16. Double quantization further reduces memory by quantizing the quantization constants themselves. This reduces total GPU memory from  $\sim 14$  GB (BF16) to approximately 5.5 GB, enabling training on a single 16 GB T4 GPU.

### 4.5 Instruction Tuning and Prompt Engineering

The model is trained on instruction-following examples from CodeAlpaca-20k. Each sample is formatted using an Alpaca-style prompt template: a system instruction, an input (code snippet or task description), and an expected output. During inference, repository context (file snippets, directory trees) is injected into the instruction field, enabling repository-aware responses without architectural changes.

### 4.6 Supervised Fine-Tuning (SFT) and Training Process

TRL's SFTTrainer implements causal language modelling loss over the target completion tokens only (loss masking on the prompt portion). Gradient accumulation over 4 steps simulates an effective batch size of 16 without requiring it in memory simultaneously. A cosine learning rate schedule with 3% warmup steps ensures smooth convergence from the pre-trained initialisation.

### 4.7 Agent Workflow

The agent is implemented as a ReAct-style loop: Reason → Act → Observe. At each turn, the model receives the conversation history plus available tool descriptions, produces a structured action (e.g., `read_file`, `list_directory`, `explain_code`), and the Python runtime executes the action and returns an observation. This continues until the model emits a final answer, keeping tool execution strictly separate from model output for auditability.

## 5. Implementation Details

---

### 5.1 Dataset Preparation

CodeAlpaca-20k is sourced from Hugging Face Datasets (sahil2801/CodeAlpaca-20k). It contains 20,022 instruction–input–output triplets covering Python, JavaScript, SQL, shell scripting, and algorithmic problem solving.

- Load with `datasets.load_dataset`, split 90/10 train/validation.
- Filter samples where the output has fewer than 20 tokens (low-quality completions).
- Deduplicate on the instruction field using exact hashing.
- Format each sample into the Alpaca prompt template and tokenize.
- Truncate to `max_seq_length = 512` with right-padding; pack short samples using TRL packing.

### 5.2 Model Selection

Gemma-2B-IT (google/gemma-2b-it) was selected for the following reasons:

- 2.18B parameters — fits within 6 GB VRAM under 4-bit quantization.
- Instruction-tuned variant provides a strong instruction-following prior.
- Apache 2.0 licence allows free academic and research use.
- Competitive coding benchmark scores relative to other sub-3B models (HumanEval pass@1  $\approx$  29% base).

### 5.3 Fine-Tuning Pipeline

The training script loads the quantized base model, attaches LoRA adapters via PEFT, and invokes TRL's SFTTrainer. Key code structure:

```
# Load 4-bit quantized base model
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True, bnb_4bit_quant_type='nf4',
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True)

model = AutoModelForCausalLM.from_pretrained(
    'google/gemma-2b-it', quantization_config=bnb_config,
    device_map='auto')

# Attach LoRA adapters
lora_cfg = LoraConfig(r=16, lora_alpha=32,
    target_modules=['q_proj', 'v_proj'],
    lora_dropout=0.05, bias='none',
    task_type=TaskType.CAUSAL_LM)

model = get_peft_model(model, lora_cfg)

# SFT Training
trainer = SFTTrainer(model=model, train_dataset=train_ds,
    max_seq_length=512, packing=True, args=training_args)
trainer.train(resume_from_checkpoint=True)
```



## 5.4 Hyperparameters

| Hyperparameter      | Value              | Rationale                                 |
|---------------------|--------------------|-------------------------------------------|
| Base Model          | google/gemma-2b-it | Compact, instruction-tuned, 2B params     |
| LoRA Rank (r)       | 16                 | Balance expressiveness vs parameter cost  |
| LoRA Alpha          | 32                 | Scaling factor = 2 x rank                 |
| LoRA Dropout        | 0.05               | Light regularization                      |
| Target Modules      | q_proj, v_proj     | Attention projection matrices             |
| Learning Rate       | 2e-4               | Standard LoRA LR range                    |
| LR Scheduler        | Cosine with warmup | Smooth decay, 3% warmup steps             |
| Batch Size          | 4 (grad accum = 4) | Effective batch 16, memory-efficient      |
| Max Sequence Length | 512 tokens         | Covers most CodeAlpaca samples            |
| Epochs              | 3                  | Sufficient for convergence on 20k samples |
| Quantization        | 4-bit NF4 (QLoRA)  | Reduces GPU memory to ~6 GB               |
| Gradient Clipping   | 1.0                | Prevents gradient explosion               |
| Optimizer           | paged_adamw_8bit   | Memory-efficient AdamW variant            |
| Checkpoint Interval | Every 200 steps    | Recovery from interruptions               |

Table 3 — Training Hyperparameters

## 5.5 Agent Implementation

The agent is built using LangChain's Tool abstraction. Three tools are registered:

| Tool                              | Description                                                       |
|-----------------------------------|-------------------------------------------------------------------|
| <code>read_file(path)</code>      | Returns the content of a specified file, truncated to 800 tokens. |
| <code>list_directory(path)</code> | Returns a formatted tree of a directory up to depth 2.            |
| <code>search_code(query)</code>   | Fuzzy-searches across loaded file contents for relevant snippets. |

## 5.6 Libraries and Frameworks

| Library / Framework       | Version | Purpose                               |
|---------------------------|---------|---------------------------------------|
| <code>transformers</code> | 4.40+   | Model loading, tokenization, training |
| <code>peft</code>         | 0.10+   | LoRA / QLoRA adapter management       |
| <code>trl</code>          | 0.8+    | SFTTrainer for supervised fine-tuning |

| Library / Framework          | Version | Purpose                                     |
|------------------------------|---------|---------------------------------------------|
| <a href="#">bitsandbytes</a> | 0.43+   | 4-bit quantization (QLoRA)                  |
| <a href="#">datasets</a>     | 2.18+   | CodeAlpaca-20k loading and preprocessing    |
| <a href="#">accelerate</a>   | 0.29+   | Distributed training and mixed precision    |
| <a href="#">langchain</a>    | 0.1+    | Agent orchestration and tool integration    |
| <a href="#">torch</a>        | 2.2+    | Core deep learning framework (CUDA backend) |
| <a href="#">evaluate</a>     | 0.4+    | Perplexity and metric computation           |

Table 4 — Libraries and Frameworks

## 6. Results and Evaluation

### 6.1 Quantitative Metrics

The evaluation suite consists of 120 tasks drawn from three difficulty tiers: 40 easy (direct function completion), 50 medium (module-level refactoring), and 30 ambiguous (design-level judgment). Human raters scored explanation clarity on a 1–5 scale across 30 sampled outputs.

| Metric                    | Base Model | Fine-tuned | Improvement |
|---------------------------|------------|------------|-------------|
| Training Loss (final)     | —          | 0.87       | —           |
| Validation Perplexity     | ~42.3      | ~18.6      | 56% lower   |
| Task Completion Rate      | 51%        | 78%        | +27 pp      |
| Code Correctness (Pass@1) | 34%        | 61%        | +27 pp      |
| Explanation Clarity       | 3.1 / 5    | 4.0 / 5    | +0.9 pts    |
| Prompt Consistency        | 62%        | 84%        | +22 pp      |
| Avg. Response Latency     | ~2.8s      | ~2.4s      | 14% faster  |

Table 5 — Performance Comparison: Base vs Fine-Tuned Model

### 6.2 Sample Outputs — Before vs After

| Prompt            | Write a Python function that reads a CSV file and returns the column names.                                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Base Model Output | Sure! Here is a Python function. Import csv. Open the file. Read it. Return                                                                                                          |
| Fine-tuned Output | <pre>import csv  def get_column_names(filepath: str) -&gt; list[str]:     with open(filepath, newline='') as f:         reader = csv.reader(f)         return next(reader, [])</pre> |

Table 6 — Sample Output Comparison

### 6.3 Observations and Analysis

- The fine-tuned model consistently produces executable, syntactically correct code for direct instruction tasks, replacing verbose filler text with focused completions.
- On medium-difficulty tasks involving existing code snippets, the model correctly identifies refactoring opportunities (e.g., replacing loops with list comprehensions) in 68% of cases.
- Ambiguous design-level tasks expose the model's capacity limits: it provides reasonable suggestions but occasionally misses cross-file dependencies not present in the context window.
- Prompt consistency improved from 62% to 84%, indicating that fine-tuning reduced stochastic hallucination and anchored the model to the instruction-following format.
- Training loss converged smoothly to 0.87 over 3 epochs with no instability after applying gradient clipping and a cosine LR schedule.

## 7. Challenges Faced and Solutions

### 7.1 Technical Challenges

| Challenge                    | Root Cause                          | Solution Applied                          |
|------------------------------|-------------------------------------|-------------------------------------------|
| <b>GPU OOM errors</b>        | Full FP32 model too large           | 4-bit QLoRA + gradient checkpointing      |
| <b>Training instability</b>  | LR too high, gradient spikes        | Reduce LR to 2e-4, clip gradients at 1.0  |
| <b>Slow convergence</b>      | Short max sequence length           | Increased seq len from 256 to 512         |
| <b>Noisy dataset samples</b> | CodeAlpaca has ~5% low-quality rows | Filtered by min token length + dedup      |
| <b>Overfitting risk</b>      | Small model + focused dataset       | LoRA dropout 0.05, early stopping         |
| <b>Kaggle session limits</b> | 90-minute GPU disconnects           | Checkpoint every 200 steps, resume script |

Table 7 — Challenges and Resolutions

### 7.2 Role of AI Tools in Development

AI assistants (Claude and ChatGPT) played a meaningful supporting role throughout the project lifecycle. Their usage is summarised below:

| Usage Area                 | Description                                                                         |
|----------------------------|-------------------------------------------------------------------------------------|
| <b>Debugging</b>           | Diagnosing cryptic CUDA OOM stack traces & identifying the exact Bits&Bytes flag.   |
| <b>Documentation</b>       | Generating docstrings and inline comments for the agent tools and training scripts. |
| <b>Hyperparameter</b>      | Discussing LoRA selection trade-offs and LR scheduler.                              |
| <b>Architecture Review</b> | Reviewing the agent prompt template for ReAct-style instruction clarity .           |
| <b>Report Writing</b>      | Structural advice on academic report formatting and section organisation.           |

## 8. Conclusion

---

### 8.1 Summary of Achievements

This project demonstrates that a compact 2B-parameter language model can be meaningfully adapted for practical coding assistance using parameter-efficient fine-tuning on consumer-grade hardware. Key achievements include:

- Successfully fine-tuned Gemma-2B using QLoRA within a single Kaggle T4 session, reducing peak VRAM usage to approximately 5.5 GB.
- Achieved a 56% reduction in validation perplexity and a 27 pp improvement in task completion rate over the base model.
- Built a functional ReAct-style coding agent with file-inspection tools, demonstrating repository-aware response generation.
- Produced a reproducible pipeline covering data preprocessing, training, evaluation, and inference, suitable for extension by future work.

### 8.2 Limitations

- The 512-token context window limits the model's ability to reason over large files or multi-file repository structures without retrieval augmentation.
- Performance on tasks requiring cross-file dependency understanding remains below frontier models.
- Evaluation was conducted on a fixed 120-task suite; broader benchmarks (HumanEval, MBPP) were not fully completed due to time constraints.

### 8.3 Future Work

- Retrieval-Augmented Generation (RAG): Integrate a vector store (FAISS or ChromaDB) so the agent can retrieve relevant code snippets from large repositories at inference time.
- Structured Diff Generation: Fine-tune on patch/diff datasets to produce line-level edit suggestions rather than full-file rewrites.
- Multi-Model Quality Control: Route ambiguous tasks to a larger frontier model and use the compact model for latency-sensitive, straightforward completions.
- Expanded Evaluation: Run on HumanEval and MBPP benchmarks with pass@1 and pass@10 metrics for standardised comparison against published models.
- Longer Context Training: Extend sequence length to 1024–2048 tokens using flash-attention and gradient checkpointing to support larger file contexts.

## 9. References

---

- [1] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. International Conference on Learning Representations (ICLR 2022). <https://arxiv.org/abs/2106.09685>
- [2] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. Advances in Neural Information Processing Systems (NeurIPS 2023). <https://arxiv.org/abs/2305.14314>
- [3] Hu, E., et al. (2023). PEFT: State-of-the-art Parameter-Efficient Fine-Tuning. Hugging Face Library. <https://github.com/huggingface/peft>
- [4] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2020). Transformers: State-of-the-Art Natural Language Processing. Proceedings of EMNLP 2020: System Demonstrations. <https://arxiv.org/abs/1910.03771>
- [5] Gemma Team, Google DeepMind (2024). Gemma: Open Models Based on Gemini Research and Technology. <https://arxiv.org/abs/2403.08295>
- [6] Chung, H. W., Hou, L., Longpre, S., Zoph, B., Tay, Y., Fedus, W., ... & Wei, J. (2022). Scaling Instruction-Finetuned Language Models. <https://arxiv.org/abs/2210.11416>
- [7] Von Werra, L., et al. (2023). TRL: Transformer Reinforcement Learning. Hugging Face Library. <https://github.com/huggingface/trl>
- [8] Chase, H. (2022). LangChain: Building Applications with LLMs through Composability. <https://github.com/langchain-ai/langchain>
- [9] Dettmers, T. (2022). BitsAndBytes: 8-bit Optimizers and Quantization Routines. <https://github.com/TimDettmers/bitsandbytes>
- [10] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems 32 (NeurIPS 2019). <https://arxiv.org/abs/1912.01703>
- [11] Sahil2801/CodeAlpaca-20k (2023). Hugging Face Dataset. <https://huggingface.co/datasets/sahil2801/CodeAlpaca-20k>
- [12] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. O., Kaplan, J., ... & Zaremba, W. (2021). Evaluating Large Language Models Trained on Code (HumanEval). <https://arxiv.org/abs/2107.03374>